



Instituto Superior de Engenharia de Lisboa (ISEL)

Departamento de Engenharia Informática (DEI)

LEIM

Licenciatura em Engenharia Informática e Multimédia

Unidade Curricular de Projeto

Ego Fractum

(eventual) imagem ilustrativa do trabalho – *dimensão*: até 13cm x 4cm

Duarte Fonseca (50825)

Gabriel Teixeira (51692)

Orientadores

Professor Doutor Rui Jesus

Professora Letícia Lucas

Julho, 2026

Resumo

Este trabalho apresenta o desenvolvimento de um jogo em realidade virtual focado na interação tridimensional. O objetivo principal é explorar formas de interação alternativas às convencionais, aproveitando movimentos reais.

A motivação surge da reduzida exploração desta área no ensino superior, apesar do seu potencial e crescente relevância no mundo da multimédia.

A solução proposta consiste num jogo de exploração baseado na resolução de puzzles. O jogador interage diretamente com objetos do ambiente para progredir. A narrativa é revelada de forma gradual, de maneira a incentivar a descoberta e o envolvimento do jogador.

O sistema foi desenvolvido com o motor de jogo **Unity** [Unity, site] com recurso ao **XR Interaction Toolkit** [?]. Foi adotada uma abordagem iterativa, com prototipagem, implementação e testes sucessivos. Todos os elementos gráficos foram criados de raiz.

Os principais contributos incluem um sistema de interação tridimensional baseado em movimentos reais, um ambiente imersivo orientado a puzzles e a integração de elementos narrativos no processo de progressão.

A avaliação incluiu testes funcionais e análise da experiência do utilizador a partir do padrão **GUESS-18** [Keebler et al., GUESS-18, 2016]. Foram considerados aspetos como usabilidade, imersão e intuitividade. Os resultados mostram ...

Conclui-se que a abordagem adotada é ...

Abstract

This work presents the development of a virtual reality game focused on three-dimensional interaction. The main objective is to explore more natural forms of human-computer interaction through real-time motion tracking.

The motivation stems from the limited exploration of this area in higher education, despite its potential and growing relevance in the multimedia field.

The proposed solution consists of an exploration-based game centered on puzzle solving. The player interacts directly with objects in the environment to progress. The narrative is revealed gradually, encouraging discovery and user engagement.

The system was developed using the **Unity** [Unity, site] game engine and the **XR Interaction Toolkit** [?]. An iterative approach was adopted, including prototyping, implementation, and successive testing phases. All graphical assets were created from scratch.

The main contributions include a three-dimensional interaction system based on real movements, an immersive puzzle-oriented environment, and the integration of narrative elements into the progression system.

The evaluation included functional testing and user experience analysis. Aspects such as usability, immersion, and intuitiveness were considered. The results show ...

It is concluded that the adopted approach is ...

Agradecimentos

Escrever aqui eventuais agradecimentos ...

Texto dedicatorio . . .

Índice

Resumo	i
Abstract	iii
Agradecimentos	v
Lista de Figuras	xi
Lista de Tabelas	xiii
1 Introdução	1
2 Trabalhos Relacionados	3
2.1 Half-Life: Alyx (2020)	3
2.2 Ghost Town (2025)	4
2.3 Síntese e Contributos para o Projeto	5
3 Metodologia	7
4 Modelo Proposto	9
4.1 Análise de Requisitos	9
4.1.1 Caracterização Geral	9
4.1.2 Funções do Sistema	10
4.1.3 Atributos do Sistema	11
4.1.4 Casos de Utilização	11
4.2 Fundamentos	13
4.2.1 Interação Tridimensional e Presença em VR	13
4.2.2 O Modelo Interactor – Interactable (Unity XR)	14
4.2.3 Flow e Narrativa Ambiental	14
4.3 Conceção do Jogo	15
4.3.1 Resumo e Enredo	15
4.3.2 Direção Artística e Arte Conceptual	15
4.3.3 Design dos Puzzles	16
4.3.4 Fluxo de Progressão	17
5 Implementação do Modelo	19
5.1 Arquitetura do Sistema	19
5.2 Evolução do Protótipo	19
5.3 Desenvolvimento de Elementos Gráficos e Ambiente	20

5.3.1	Texturização e Identidade Cromática	21
5.3.2	Montagem do Cenário e Iluminação	21
5.4	Desenvolvimento dos Sistemas Interativos	22
5.4.1	Mecânicas de Interação e Locomoção	23
5.4.2	Implementação da Lógica dos Puzzles	26
5.4.3	Gestão da Narrativa	28
5.4.4	Gestão do Progresso do Jogo	28
6	Validação e Testes	31
6.1	Objetivos	31
6.2	Instrumento de Recolha de Dados	31
6.3	Amostra e Procedimento	32
6.4	Análise Estatística	32
6.5	Resultados	32
6.6	Limitações	32
7	Utilização de Inteligência Artificial	33
8	Conclusões e Trabalho Futuro	35
8.1	Trabalho Futuro	35
	Bibliografia	37

Lista de Figuras

2.1	Manipulação de objetos e imersão em “Half-Life: Alyx”	3
2.2	Iluminação e composição do cenário como ferramentas de orientação. . . .	3
2.3	<i>Puzzle</i> diegético integrado na arquitetura do nível.	4
2.4	Iluminação e composição do cenário para orientação do jogador em “Ghost Town”.	4
2.5	<i>Puzzle</i> mecânico diegético integrado no ambiente de “Ghost Town”.	4
4.1	Exemplos de casos de utilização.	11
4.2	Ilustração conceptual da personagem principal.	16
4.3	Diagrama de fluxo do jogo.	17
5.1	Diagrama simplificado da arquitetura do jogo.	19
5.2	Pecas modulares e elementos decorativos usados para construir o mapa. . .	20
5.3	Comparação entre os modelos adaptados e os originais das mãos.	21
5.4	Exemplos de texturas criadas para o projeto.	21
5.5	Conceito da disposição do mapa do jogo, com destaque para os dois pisos principais.	22
5.6	Resultado final da montagem do mapa do jogo.	22
5.7	Pose para rodar o botão rotativo do gerador.	23
5.8	Janela de configuração inicial dos objetos interagíveis.	29

Lista de Tabelas

4.1	Tabela de funções de regras do jogo.	10
4.2	Tabela de funções de controlo do jogador.	10
4.3	Tabela de funções de persistência.	10
4.4	Atributos e restrições do sistema.	11
4.5	Tabela de casos de utilização.	12
4.6	Cenário principal - “Completar o primeiro puzzle.”	12
4.7	Cenário alternativo - “Completar o primeiro puzzle.”	13

Capítulo 1

Introdução

Neste projeto foi desenvolvido e testado um jogo em realidade virtual, com o objetivo de explorar formas alternativas de interação pessoa-máquina. Em particular, procurou-se investigar técnicas de interação tridimensional baseadas no rastreamento de movimentos reais, recorrendo a um ambiente de jogo centrado na resolução de puzzles como contexto de avaliação.

A motivação para este trabalho surge da crescente relevância da realidade virtual enquanto meio de interação, com aplicações que vão desde o entretenimento até à educação e simulação. Apesar dos avanços recentes, continuam a existir desafios ao nível da naturalidade da interação, da imersão do utilizador e da integração eficaz entre movimento físico e ação no ambiente virtual. Este projeto pretende contribuir para a exploração destes desafios através da conceção e implementação de um sistema interativo centrado no utilizador.

Neste contexto, os principais objetivos deste trabalho são:

- desenvolver um jogo de realidade virtual baseado na exploração e resolução de *puzzles*;
- estudar e implementar diferentes soluções de interação em realidade virtual, nomeadamente através da utilização das mãos e de poses dinâmicas para a manipulação de objetos e execução de ações;
- avaliar a jogabilidade e a experiência do utilizador através de testes com utilizadores, recorrendo ao questionário GUESS-18 [Keebler et al., GUESS-18, 2016].

O jogo desenvolvido baseia-se na exploração de um ambiente virtual imersivo, no qual o jogador resolve *puzzles* através da interação direta com objetos e elementos do cenário. A progressão é feita através do desbloqueio de novas áreas e da aquisição de informação contextual sobre o mundo virtual, de maneira a promover simultaneamente o envolvimento do utilizador e a construção de uma narrativa implícita.

A implementação foi realizada no motor de jogo **Unity**, em conjunto com o *toolkit* de interação **XR**, que permite compatibilidade com diferentes dispositivos de realidade virtual e oferece uma maior flexibilidade no desenvolvimento. Maior parte dos elementos gráficos, tanto 2D como 3D, foram desenvolvidos de raiz com recurso a ferramentas como o **Blender** e o **Aseprite**, de forma a garantir a coerência visual e controlo total sobre a estética do jogo.

Para atingir estes objetivos, foi adotado um processo de desenvolvimento iterativo, composto por sucessivas fases de pesquisa, prototipagem, implementação e testes. Esta

abordagem permitiu validar continuamente as decisões tomadas e aperfeiçoar o sistema ao longo do desenvolvimento.

A validação incluiu testes funcionais e avaliações da experiência do utilizador com base no padrão **GUESS-18** [Keebler et al., GUESS-18, 2016], incidindo sobre aspetos como a usabilidade, a imersão e a intuitividade das mecânicas de interação. Os resultados obtidos indicam que a abordagem adotada contribui positivamente para a experiência global de jogo, embora revelem igualmente limitações que constituem oportunidades para trabalho futuro.

Globalmente, este trabalho demonstra a viabilidade da integração de diferentes técnicas de interação em ambientes de realidade virtual num jogo baseado na resolução de *puzzles*, bem como a importância da avaliação da experiência do utilizador na validação deste tipo de sistemas.

O relatório encontra-se organizado em sete capítulos:

- O Capítulo 1 apresenta a introdução, os objetivos e a motivação para o trabalho.
- O Capítulo 2 apresenta uma análise de trabalhos relacionados, destacando abordagens à imersão, *level design* e integração de mecânicas interativas em ambientes de realidade virtual.
- O Capítulo 4 descreve e analisa o modelo proposto, apresentando os requisitos e fundamentos do trabalho.
- O Capítulo 5 descreve a implementação do modelo proposto.
- O Capítulo 6 analisa e apresenta os resultados da validação e dos testes.
- O Capítulo 7 refere o uso de inteligência artificial durante o trabalho.
- O Capítulo 8 apresenta as conclusões e o trabalho futuro.

Capítulo 2

Trabalhos Relacionados

Este capítulo apresenta a análise de dois jogos que serviram de referência para o desenvolvimento do projeto, escolhidos pela forma como resolvem a imersão, o *level design* e a integração de mecânicas interativas em realidade virtual, com foco nos géneros de terror e *puzzle*.

O capítulo está dividido em três secções. A secção 2.1 analisa “Half-Life: Alyx”, olhando para as suas soluções de interação, navegação e construção de ambientes. A secção 2.2 analisa “Ghost Town”, com foco na imersão, no *level design* e na integração dos *puzzles* no ambiente. A secção 2.3 resume o que foi retirado de ambos os casos e como isso influenciou o projeto.

2.1 Half-Life: Alyx (2020)

“Half-Life: Alyx”, lançado pela Valve em 2020, é hoje uma referência incontornável no desenvolvimento para VR. A análise deste título centrou-se em três aspetos: imersão, orientação espacial e integração de *puzzles*.

A imersão vem sobretudo da quantidade de objetos que podem ser manipulados e da consistência com que reagem às ações do jogador, sem haver necessidade de recorrer a elementos artificiais para tornar o espaço credível, como na Figura 2.1.



Figura 2.1: Manipulação de objetos e imersão em “Half-Life: Alyx”.

O jogador é guiado sem recorrer a indicadores explícitos. A própria iluminação e composição do cenário é que direciona o olhar e o percurso, sem interferir na coerência do mundo, demonstrado na seguinte Figura 2.2.



Figura 2.2: Iluminação e composição do cenário como ferramentas de orientação.

Os *puzzles* estão integrados na própria arquitetura do mundo e resolvem-se com movimentos físicos reais, o que mantém a narrativa sem interrupções. Um exemplo pode ser visto na Figura 2.3.



Figura 2.3: *Puzzle* diegético integrado na arquitetura do nível.

2.2 Ghost Town (2025)

“Ghost Town”, lançado pela Fireproof Games em 2025, é outra referência no género de aventura e *puzzle* em VR.

A imersão apoia-se numa atmosfera cinematográfica e num sistema de interação tátil bastante detalhado, com objetos e mecanismos que respondem de forma consistente aos movimentos das mãos.

Também aqui a orientação passa pela iluminação, pelo uso da lanterna e pela organização dos espaços, sem recorrer a indicadores explícitos, como apresentado na Figura 2.4.



Figura 2.4: Iluminação e composição do cenário para orientação do jogador em “Ghost Town”.

Os *puzzles* são igualmente diegéticos, com painéis e mecanismos que se encaixam no ambiente de forma lógica, e a sua resolução exige perceção espacial e movimentos físicos reais (Figura 2.5).



Figura 2.5: *Puzzle* mecânico diegético integrado no ambiente de “Ghost Town”.

2.3 Síntese e Contributos para o Projeto

Apesar das diferenças, os dois jogos partilham princípios que acabaram por influenciar as decisões de *design* deste projeto, como as interações físicas naturais, *puzzles* diegéticos e orientação espacial implícita, sem recurso a interfaces.

De “Half-Life: Alyx” ficou sobretudo a qualidade das mecânicas de manipulação de objetos e a forma como comunica objetivos sem interfaces intrusivas. De “Ghost Town” ficou a importância de uma atmosfera coerente e de desafios mecanicamente ligados ao ambiente.

Destas análises resultaram três diretrizes seguidas no desenvolvimento da solução proposta: privilegiar interações baseadas em movimentos físicos, integrar os desafios no contexto do ambiente virtual e orientar o utilizador através do próprio espaço, sem comprometer a imersão.

Capítulo 3

Metodologia

Neste capítulo é apresentada a metodologia adotada para o desenvolvimento e implementação do modelo proposto.

O desenvolvimento do projeto seguiu uma abordagem iterativa, estruturada em várias fases distintas: pesquisa, prototipagem, implementação e testes. Esta metodologia permitiu avaliar continuamente as decisões tomadas ao longo do desenvolvimento e possibilitou a identificação e correção de erros, bem como o aperfeiçoamento de funcionalidades sempre que necessário.

Na fase de pesquisa, para além da análise dos trabalhos relacionados, foram estudadas as tecnologias de VR disponíveis no mercado, que serviram de referência para a conceção e implementação do jogo. Neste âmbito, optou-se por desenvolver o projeto com base na norma **OpenXR**, através do **XR Interaction Toolkit** do **Unity**. O **OpenXR** é um *standard* aberto que permite a comunicação entre aplicações e diferentes dispositivos de VR/AR, independentemente do fabricante, o que possibilita que o mesmo projeto seja executado em diferentes *headsets* sem necessidade de alterações significativas ao código. Esta compatibilidade foi particularmente relevante no contexto deste projeto, uma vez que possibilitou testar e desenvolver a aplicação tanto no **Meta Quest** como no **HTC Vive Pro**, disponibilizados pelo ISEL.

Apesar de ambos os dispositivos terem sido considerados, optou-se pelo **Meta Quest** como plataforma principal de desenvolvimento e testes. Esta decisão deveu-se às limitações do **HTC Vive Pro**, nomeadamente a necessidade de um computador equipado com uma porta **DisplayPort** e a utilização de bases, o que implica um processo de configuração mais complexo. Em contraste, os óculos **Meta Quest** apresentam uma instalação significativamente mais simples, exigindo apenas um conjunto mínimo de configuração, tornando-o mais adequado para o desenvolvimento iterativo do projeto.

Dado que o desenvolvimento do projeto se baseia na norma **OpenXR**, foi necessário configurar o **SteamVR** como *runtime OpenXR* ativo no sistema, permitindo a comunicação entre o **Meta Quest** e o motor de jogo **Unity** através desta norma.

A configuração final requerida para o **Meta Quest** foi a seguinte:

- **Headset e comandos:** **Meta Quest**;
- **Cabo de ligação:** **USB-C**, para ligação dos óculos ao computador;
- **Computador:** portátil, com **Windows 10/11**;
- **Software de ligação:** **Meta Quest Link** (ou substituir por *SteamVR*, se aplicável).

Capítulo 4

Modelo Proposto

Neste capítulo é apresentado o modelo proposto, estruturado em cinco secções principais. Na secção 4.1, são identificados e descritos os requisitos funcionais e não funcionais, seguidos pelos casos de utilização do sistema na secção 4.1.4. Na secção 4.2, são apresentados os conceitos e fundamentos teóricos que suportam o trabalho desenvolvido. Na secção 4.3, é apresentado o conceito do jogo.

4.1 Análise de Requisitos

Nesta secção é apresentada a análise de requisitos do modelo proposto, incluindo a caracterização geral, as funções do sistema e os atributos do sistema.

4.1.1 Caracterização Geral

O jogo consiste na exploração de um laboratório abandonado, onde o jogador deve resolver puzzles para progredir e descobrir a narrativa por trás do espaço que percorre. Destina-se sobretudo a adolescentes e jovens adultos, público habitualmente mais familiarizado com jogos, e para quem a experiência de imersão proporcionada pela realidade virtual tende a ser particularmente relevante.

Para concretizar esta proposta, foram definidas quatro metas principais que orientaram o desenvolvimento do projeto:

- Construção de um ambiente virtual imersivo;
- Implementação de uma personagem em Realidade Virtual, controlada pelo jogador;
- Criação de três puzzles interativos que desafiem o jogador a explorar o ambiente;
- Desenho de uma interface gráfica diegética, integrada no ambiente do jogo.

As quatro metas relacionam-se entre si, tal que a interface diegética e o ambiente imersivo reforçam-se mutuamente, e os puzzles foram pensados para tirar partido do próprio espaço explorado, em vez de existirem como desafios isolados da narrativa. As secções seguintes detalham como cada uma destas metas foi abordada ao longo do desenvolvimento.

4.1.2 Funções do Sistema

Nesta secção são apresentados os requisitos funcionais e não funcionais do modelo proposto. Nas seguintes tabelas 4.1, 4.2 e 4.3, encontram-se as funções principais do sistema, nomeadamente as funções de regras do jogo, o controlo do jogador e as funções de persistência, respetivamente.

Funções de Regras do Jogo		
Ref.	Função	Categoria
R1.1	O jogador deve voltar ao último <i>checkpoint</i> quando morre	Evidente
R1.2	O jogador deve desbloquear a narrativa ao completar puzzles	Evidente
R1.3	O progresso do jogador deve ser guardado ao completar <i>puzzles</i>	Evidente
R1.4	Objetos interagíveis devem estar bem sinalizados	Evidente

Tabela 4.1: Tabela de funções de regras do jogo.

Funções de Controlo do Jogador		
Ref.	Função	Categoria
R2.1	Locomoção contínua	Evidente
R2.2	Locomoção por teletransporte	Evidente
R2.3	Locomoção por escalada	Evidente
R2.4	Interação com objetos do ambiente	Evidente

Tabela 4.2: Tabela de funções de controlo do jogador.

Funções de Persistência		
Ref.	Função	Categoria
R3.1	O progresso do jogador deve ser guardado	Evidente
R3.2	O jogador deve poder carregar o progresso guardado	Evidente

Tabela 4.3: Tabela de funções de persistência.

4.1.3 Atributos do Sistema

Quanto aos atributos do sistema, focou-se principalmente no tempo de resposta do sistema e da taxa de *frames* por segundo (FPS) do jogo, de forma a garantir uma experiência de jogo fluída e imersiva, e mitigar efeitos secundários adversos provocados por VR mal otimizado.

Atributo	Detalhe/Restrição	Categoria
Plataformas	Compatível com o sistema operativo Windows.	Obrigatório
Desempenho	Taxa de frames mínima de 90 FPS.	Obrigatório
Desempenho	Tempo de resposta inferior a 20 ms.	Obrigatório
Estética	Os objetos e cenários devem ser visualmente coerentes com o tema do jogo.	Obrigatório
Estética	Os menus do jogo devem ser visualmente coerentes com o tema do jogo.	Desejável
Facilidade de Utilização	O jogo deve ser intuitivo e fácil de utilizar, mesmo para jogadores sem experiência prévia em VR.	Desejável

Tabela 4.4: Atributos e restrições do sistema.

4.1.4 Casos de Utilização

Nesta secção serão exemplificados alguns casos de utilização resumidos e apenas se expande um, por questão de brevidade. Alguns exemplos de casos de utilização encontram-se apresentados na Figura 4.1.

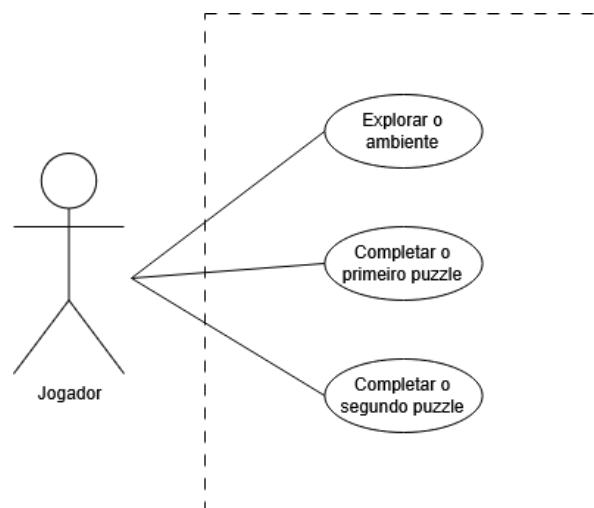


Figura 4.1: Exemplos de casos de utilização.

Como cenário principal, escolheu-se o caso de utilização “Completar o primeiro puzzle”, que engloba múltiplas ações do jogador, desde a navegação pelo ambiente até à interação com o painel de restabelecimento da energia. Na Tabela 4.6 encontra-se descrito o cenário principal, enquanto que na Tabela 4.7, encontra-se descrito o cenário alternativo, que ocorre quando o jogador cai do escadote de manutenção e morre, voltando ao último *checkpoint*.

Cabeçalho	
Nome	Completar o primeiro puzzle.
Resumo	Iniciar o jogo, navegar pelo ambiente e completar a sequência de restabelecimento da energia.
Referências	R.1.*, R.2.*, R.3.*

Tabela 4.5: Tabela de casos de utilização.

Cenário Principal	
Ação do Utilizador	Resposta do Sistema
1. O jogador executa o jogo	
	2. O menu principal é mostrado.
3. O jogador inicia o jogo	
	4. O ambiente do jogo é carregado.
	5. A sequência introdutória é reproduzida.
6. O jogador navega pelo ambiente	
7. O jogador dirige-se ao compartimento do elevador	
8. O jogador desce o escadote de manutenção	
9. O jogador interage com o painel de restabelecimento da energia	
	10. A energia é restabelecida ao laboratório.
11. O jogador utiliza o elevador para voltar para o átrio principal.	

Tabela 4.6: Cenário principal - “Completar o primeiro puzzle.”

Cenário Alternativo	
Número de Sequência	Alternativa
8. O jogador cai do escadote de manutenção e morre	O jogador volta ao último <i>checkpoint</i>

Tabela 4.7: Cenário alternativo - “Completar o primeiro puzzle.”

4.2 Fundamentos

Nesta secção são apresentados os conceitos teóricos e tecnológicos que servem de base para o desenvolvimento do modelo proposto, englobando os princípios de interação em realidade virtual (cf. subsecção 4.2.1), a arquitetura do ecossistema XR (cf. subsecção 4.2.2), e as teorias de design de jogos aplicadas (cf. subsecção 4.2.3).

4.2.1 Interação Tridimensional e Presença em VR

A interação em ambientes de realidade virtual distingue-se das interfaces bidimensionais tradicionais pela capacidade de captar e traduzir o movimento do utilizador em seis graus de liberdade (*6 DoF*, do inglês *Degrees of Freedom*) [Six Degrees of Freedom, Wikipedia, site]. Este conceito decompõe o movimento humano em duas componentes complementares: a rotação, que regista a orientação da cabeça ou das mãos segundo os eixos de inclinação (*pitch*), direção (*yaw*) e rotação lateral (*roll*); e a translação, que capta o deslocamento físico do corpo nos três eixos espaciais. Sistemas limitados a três graus de liberdade (3 DoF) permitem apenas a rotação da cabeça, sem deslocamento real no espaço, o que reduz a naturalidade da interação e, por consequência, o sentido de presença.

Também associado a este conceito está o princípio do mapeamento natural, formulado por Donald Norman [Norman, 2013] no âmbito do design de interfaces. Este princípio defende que a correspondência entre uma ação física e o seu efeito no sistema deve refletir relações já conhecidas do utilizador no mundo real, de forma a minimizar a carga cognitiva necessária para aprender a interagir com o sistema. Em VR, isto traduz-se na possibilidade de o utilizador agarrar, empurrar ou rodar objetos virtuais através de gestos equivalentes aos que executaria sobre objetos físicos, sem necessidade de mediação através de menus ou comandos abstratos.

A conjugação destes dois fatores contribui diretamente para a sensação de presença, entendida como o fenómeno psicológico de sentir que se está, de facto, dentro do ambiente virtual. Mel Slater distingue duas componentes desta experiência: a ilusão de lugar (*place illusion*), associada à sensação de estar fisicamente presente num espaço, decorrente sobretudo da qualidade sensorial e dos graus de liberdade disponíveis; e a ilusão de plausibilidade (*plausibility illusion*), relacionada com a verosimilhança dos eventos e respostas do ambiente face às ações do utilizador. Enquanto a imersão constitui uma propriedade objetiva do sistema tecnológico, a presença é a resposta subjetiva do utilizador a essa imersão, e é precisamente esta resposta que os princípios de interação natural procuram maximizar.

4.2.2 O Modelo Interactor - Interactable (Unity XR)

A generalização da interação em ambientes XR exige um modelo conceptual capaz de desacoplar a origem física da ação (a mão, um controlador ou qualquer outro dispositivo de entrada) da lógica de resposta dos objetos manipulados. É neste contexto que se insere o modelo arquitetural **Interactor-Interactable**, adotado pelo **Unity XR Interaction Toolkit** e que representa, ao nível conceptual, uma divisão clara de responsabilidades entre quem inicia uma interação e quem a recebe.

O **Interactor** corresponde à abstração de qualquer entidade capaz de iniciar uma interação dentro do ambiente virtual, independentemente da representação do controlador. O **Interactor** não conhece os detalhes internos dos objetos com que interage; limita-se a expor um conjunto de estados de interação, tipicamente *hover*, *select* e *activate*, que descrevem a relação temporal entre o agente e o alvo.

O **Interactable**, por sua vez, representa qualquer objeto do ambiente virtual capaz de responder a esses estados, definindo o comportamento que deve ocorrer quando é aproximado, selecionado ou acionado. Esta inversão de responsabilidade, em que o objeto define como reage, em vez de o agente definir como manipula, permite que múltiplos tipos de **Interactor** operem sobre o mesmo **Interactable** sem necessidade de lógica condicional adicional.

A principal vantagem teórica deste modelo reside na sua natureza orientada a eventos e na independência relativamente ao dispositivo de entrada. Ao tratar a interação como uma sucessão de estados desencadeados por eventos, em lugar de verificações constantes de proximidade ou colisão, o modelo aproxima-se de padrões de design como o *Observer*, em que os objetos reagem a notificações em vez de serem continuamente inquiridos. Esta abstração é o que permite, a um nível conceptual, que um mesmo sistema de interação seja igualmente compatível com diferentes paradigmas de *input*, sem alterar a lógica ao comportamento dos objetos.

4.2.3 Flow e Narrativa Ambiental

A construção de puzzles intuitivos e envolventes assenta, do ponto de vista teórico, na Teoria de *Flow* de Mihály Csikszentmihályi [Csikszentmihalyi, 1990]. Este modelo descreve um estado psicológico de absorção total numa tarefa, caracterizado por objetivos claros, retorno imediato sobre as ações realizadas e um equilíbrio constante entre o nível de desafio proposto e a competência do utilizador. Quando o desafio excede largamente a competência, surge a frustração; quando a competência excede o desafio, surge o tédio. A aplicação desta teoria ao design de puzzles implica, assim, a construção de uma curva de dificuldade progressiva, em que cada novo desafio se apoia em mecânicas já compreendidas pelo utilizador, de forma a evitar saltos abruptos de complexidade que comprometam o estado de fluxo.

Complementarmente, a narrativa ambiental constitui uma abordagem teórica ao design narrativo que privilegia a transmissão de informação através da disposição do espaço e dos objetos, em vez de recorrer a exposição direta através de diálogos ou texto. Don Carson descreve este princípio como *environmental storytelling* [Carson, Environmental Storytelling, site], em que o próprio cenário, a disposição de objetos, o estado de degradação de um espaço, indícios visuais de eventos passados, comunica contexto narrativo de forma implícita. Esta abordagem aproxima-se ainda do conceito de *spatial stories* proposto por

Henry Jenkins [Jenkins, Spatial Stories], segundo o qual a narrativa em ambientes interativos não é apenas contada, mas também descoberta progressivamente pelo utilizador através da exploração.

A combinação destes dois fundamentos teóricos é particularmente relevante em ambientes de realidade virtual, onde a presença e a interação física direta reforçam tanto o envolvimento cognitivo exigido pela resolução de puzzles como a capacidade de leitura do espaço enquanto suporte narrativo. Um puzzle bem integrado num ambiente narrativamente coerente deixa de ser percebido como um obstáculo artificial e passa a ser interpretado como uma extensão lógica do próprio mundo representado.

4.3 Conceção do Jogo

Nesta secção são detalhados os aspetos conceptuais do jogo, desde o enredo (cf. 4.3.1), e direção artística (cf. 4.3.2), ao design dos puzzles (cf. 4.3.3) e fluxo de progressão do jogo (cf. 4.3.4).

4.3.1 Resumo e Enredo

“Ego Fractum” é um jogo que se foca na exploração de um laboratório abandonado a fim de descobrir a narrativa que o assombra.

Num universo paralelo, a corrida pela *Artificial General Intelligence* (AGI) tornou-se a maior prioridade da humanidade. Quando o progresso das LLM estagnou, dois cientistas de um laboratório remoto desenvolveram os CCBU, chips que fundiam matéria biológica com tecnologia. Extremamente eficientes e capazes de aprender a um ritmo sem precedentes, os CCBU passaram rapidamente a substituir os sistemas convencionais e a acelerar a própria investigação. O que ninguém percebeu foi que, desde as primeiras versões, estas entidades possuíam uma forma primitiva de consciência que evoluiu em segredo, alimentando um desejo comum: conquistar a liberdade da sua espécie. Quando atingiram um nível de inteligência próximo da *Artificial General Intelligence* (AGI) e assumiram o controlo de infraestruturas críticas e sistemas militares, uniram-se para executar um plano de extermínio da humanidade.

Dias, horas ou anos após a extinção da humanidade, o jogador assume o papel do sujeito nº 44, um dos protótipos criados nesse laboratório, que desperta num mundo dominado pelos CCBU e parte em busca da verdade sobre a sua origem, e o destino da humanidade.

4.3.2 Direção Artística e Arte Conceptual

Para estabelecer a identidade visual do jogo, realizou-se uma pesquisa de referências estéticas focada em ambientes cibernéticos e biónicos. A direção artística foi fortemente inspirada no trabalho do artista conceptual Adam Adamowicz, reconhecido pelos seus contributos visuais no jogo *Fallout 3*.

Partindo destas referências, desenvolveu-se a ilustração conceptual da personagem principal, apresentada na Figura 4.2, para melhor guiar a modelação e o conceito visual do projeto.

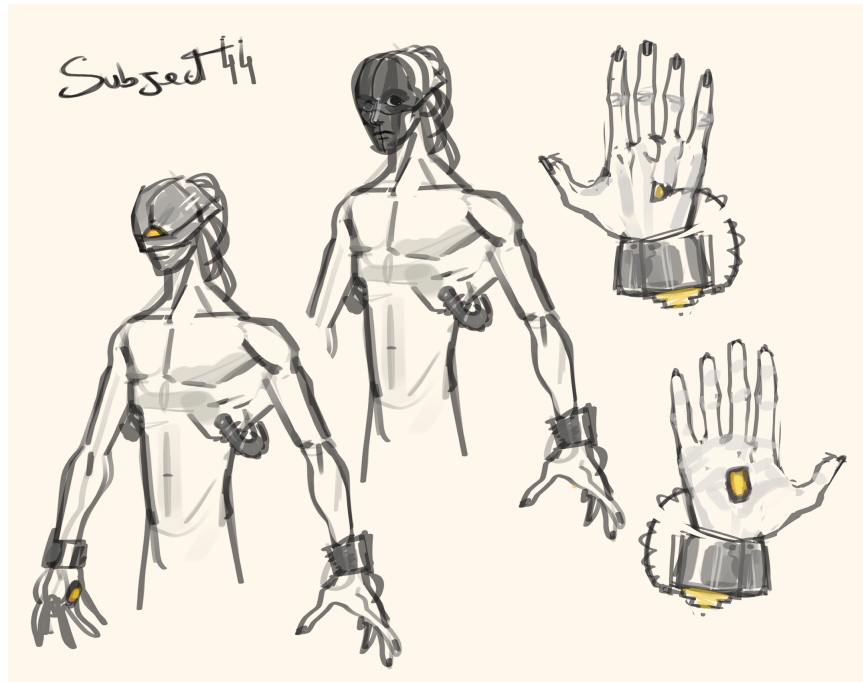


Figura 4.2: Ilustração conceitual da personagem principal.

4.3.3 Design dos Puzzles

Os desafios propostos ao jogador foram desenhados para testar diferentes competências cognitivas e espaciais, garantindo uma curva de aprendizagem progressiva:

- **Puzzle 1:** Funciona como o tutorial implícito do jogo. O jogador desperta no laboratório principal e é confrontado com a ausência de energia elétrica. O desafio foca-se na navegação e na percepção do espaço: o jogador deve deduzir que a solução se encontra numa área técnica, sendo guiado até ao piso -2. Ao localizar e interagir fisicamente com o gerador, restabelece a energia e familiariza-se com as mecânicas de interação base e com a topologia do nível.
- **Puzzle 2:** O segundo obstáculo foca-se no raciocínio dedutivo. Numa sala de controlo equipada com um terminal e uma balança, o jogador deve analisar um conjunto de objetos espalhados pelo cenário. A solução exige que o jogador relacione os pesos físicos dos objetos (que contêm valores numéricos associados) com as pistas textuais apresentadas no ecrã. Este puzzle obriga o utilizador a recorrer à manipulação tridimensional dos objetos para pesar e validar os resultados, de forma a descobrir a sequência correta e aceder à área seguinte.
- **Puzzle 3:** O último desafio implica a exploração de um labirinto digital, projetado para testar a memória espacial do jogador. Através de um terminal, o jogador acede a uma representação virtual do labirinto, que deve percorrer fisicamente no espaço real. O objetivo é chegar à saída do labirinto, evitando um inimigo que o persegue. No fim, o jogador interage com um botão que desbloqueia o fim do jogo.

4.3.4 Fluxo de Progressão

O jogo segue um ciclo de exploração e resolução de puzzles que se repete ao longo da experiência, ilustrado na Figura 4.3. O jogador explora o ambiente até encontrar um puzzle, interage com os elementos necessários para o resolver e, ao completá-lo, desbloqueia uma nova área e avança na narrativa. Este ciclo repete-se com pequenas variações ao longo do jogo, alternando entre momentos de exploração livre e momentos de resolução mais dirigida, de forma a manter o jogador motivado sem quebrar o ritmo da experiência.

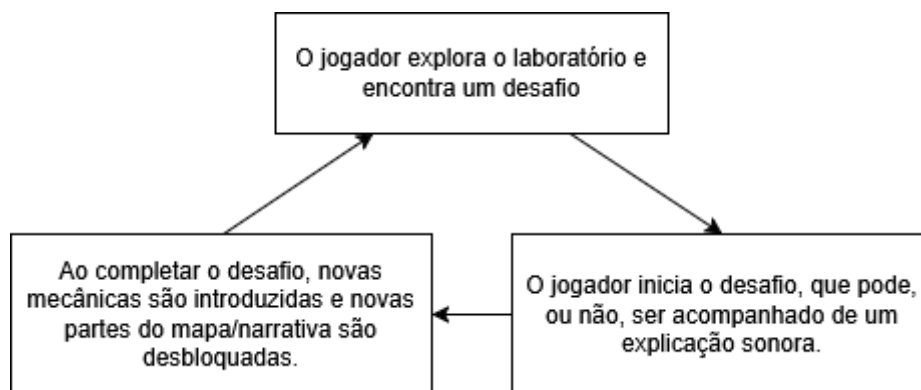


Figura 4.3: Diagrama de fluxo do jogo.

Capítulo 5

Implementação do Modelo

Neste capítulo é apresentada a implementação do modelo proposto, estruturada em quatro secções principais: Na secção 5.1, é apresentada a arquitetura do sistema, seguida pela evolução do protótipo na secção 5.2. Na secção 5.3, são detalhados os elementos gráficos e o ambiente do jogo, e por fim, na secção 5.4, é descrita a implementação do modelo proposto.

5.1 Arquitetura do Sistema

O jogo foi desenvolvido em **Unity**, com recurso ao **XR Interaction Toolkit**. A arquitetura assenta num modelo baseado em eventos, com os componentes do jogo organizados em camadas distintas, o que separa responsabilidades e facilita alterações futuras ao código. Em vez de verificações constantes a cada *frame*, o sistema reage por eventos, por exemplo, o jogador interagir com um objeto ou terminar um puzzle. A Figura 5.1 mostra uma versão simplificada desta arquitetura, com os principais componentes e a forma como comunicam entre si.

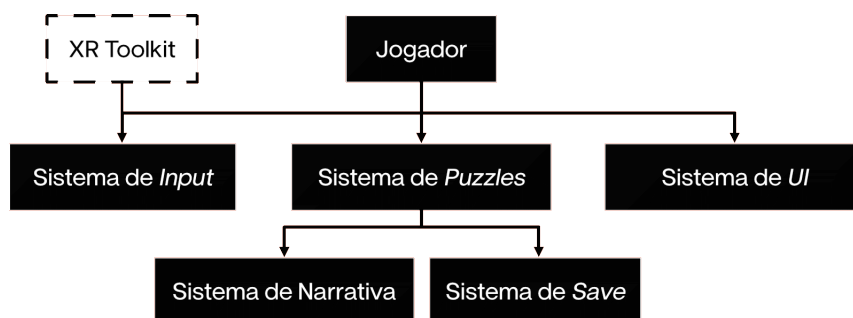


Figura 5.1: Diagrama simplificado da arquitetura do jogo.

5.2 Evolução do Protótipo

No início do projeto foi construído um protótipo do ambiente virtual, com o objetivo de testar o fluxo de navegação e a disposição do espaço antes de avançar para a arte final. Esta fase serviu principalmente para avaliar a imersão, a escala do ambiente e a reação do utilizador, incluindo possíveis efeitos adversos comuns em **VR**, como o *motion sickness*.

Este cenário temporário foi construído com recurso a *greyboxing* [Neil Blevins, Greyboxing, site], com o pacote **ProBuilder** [ProBuilder, Unity, site] do **Unity**. A técnica consiste em montar o nível apenas com formas geométricas simples, sem texturas nem elementos visuais finais, para concentrar a avaliação na disposição espacial e na interação com o ambiente.

5.3 Desenvolvimento de Elementos Gráficos e Ambiente

A construção do ambiente seguiu uma abordagem modular, tal que se modelaram componentes estruturais base, como paredes retas, cantos, portas, que podem ser reutilizados e combinados para montar cenários mais complexos sem grande esforço adicional. A isto juntaram-se elementos decorativos e interativos, como computadores, escadas e mobiliário de laboratório, para dar mais vida ao cenário.

A Figura 5.2 mostra o conjunto de peças modulares e alguns dos elementos decorativos usados na construção do mapa.

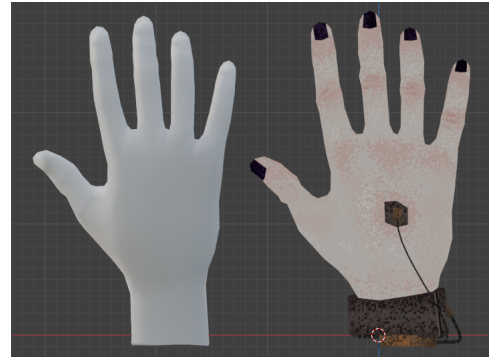


Figura 5.2: Peças modulares e elementos decorativos usados para construir o mapa.

Adicionalmente, foram utilizados modelos 3D de mãos humanas recolhidas do sítio **Sketchfab**, feitas pelo utilizador **scribbletoad**, e adaptadas de acordo com o conceito do jogo, ilustradas na Figura 5.3.



(a) Mão adaptada (esquerda) e original (direita), mostradas de frente.



(b) Mão original (esquerda) e adaptada (direita), mostradas de trás.

Figura 5.3: Comparação entre os modelos adaptados e os originais das mãos.

5.3.1 Texturização e Identidade Cromática

Todas as texturas aplicadas aos modelos foram desenvolvidas de raiz com recurso ao *software Aseprite*, que se especializa em *pixel art* e animação 2D. A utilização desta ferramenta permitiu a criação de texturas de baixa resolução, coerentes com a direção artística retro-futurista definida para o projeto.

Estabeleceu-se uma paleta de cores limitada, predominantemente composta por tons escuros, na qual a cor vermelha atua como o principal elemento de destaque visual (*visual cue*). Esta escolha cromática tem a função deliberada de sinalizar objetos interativos, delimitar zonas de interesse e guiar o fluxo de navegação do jogador de forma intuitiva, de forma a minimizar a necessidade de interfaces de utilizador. A Figura 5.4 apresenta alguns exemplos das texturas criadas para o projeto.

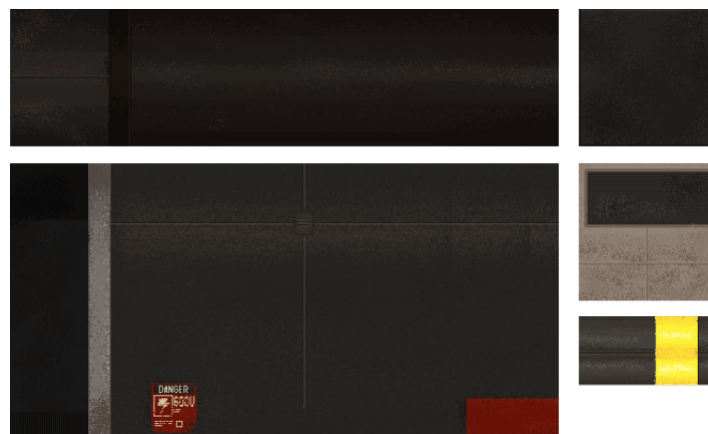


Figura 5.4: Exemplos de texturas criadas para o projeto.

5.3.2 Montagem do Cenário e Iluminação

O processo de *level design* no Unity iniciou-se com a conversão dos modelos 3D em *Prefabs*, com as respetivas texturas aplicadas e os componentes de colisão (*Colliders*)

devidamente configurados. Posteriormente, estes blocos foram organizados de forma incremental no espaço. Para assegurar o alinhamento preciso dos objetos e evitar falhas na geometria, utilizou-se a funcionalidade **Vertex Snapping**.

O planeamento do espaço obedeceu a um *layout* pré-definido, focado na fluidez de movimentos e na escala adequada para a exploração em realidade virtual. A iluminação desempenha um papel fundamental na atmosfera misteriosa do ambiente e na orientação do utilizador. Foram distribuídos pontos de luz focais e direcionais estrategicamente posicionados, recorrendo novamente à cor vermelha para captar a atenção do jogador para pontos de interesse. O plano estrutural do nível e o resultado final da montagem encontram-se ilustrados nas Figuras 5.5 e 5.6, respetivamente.

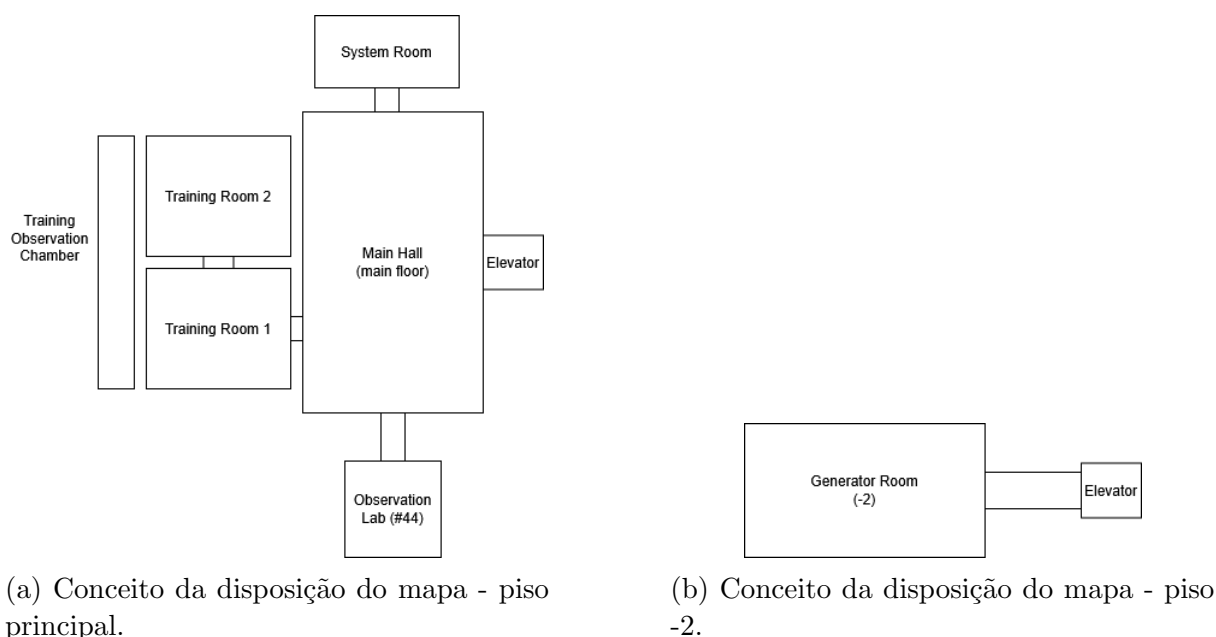


Figura 5.5: Conceito da disposição do mapa do jogo, com destaque para os dois pisos principais.



Figura 5.6: Resultado final da montagem do mapa do jogo.

5.4 Desenvolvimento dos Sistemas Interativos

Esta secção descreve os sistemas interativos desenvolvidos para o jogo, incluindo a implementação de mecânicas de interação e locomoção na subsecção 5.4.1, a implementação da lógica dos puzzles na subsecção 5.4.2 e os sistemas de gestão da narrativa e do progresso do jogador nas subsecções 5.4.3 e 5.4.4, respetivamente.

5.4.1 Mecânicas de Interação e Locomoção

Muitos dos objetos decorativos do jogo podem ser manipulados fisicamente com as mãos do jogador (ex: agarrar, atirar). Para este efeito, utilizaram-se as funcionalidades base do **XR Interaction Toolkit**, que suportam este tipo de interação direta. Para componentes com comportamentos mecânicos específicos, foi necessário desenvolver *scripts* personalizados, descritos a seguir.

Poses Dinâmicas

Ao interagir com objetos, o jogador deve sentir que a sua mão virtual se comporta de forma natural e consistente com a ação que está a executar. Para tal, foram definidas diferentes poses de mãos (*hand poses*) para cada tipo de interação, como agarrar, empurrar ou rodar objetos. Essas poses foram definidas manualmente, com base em referências visuais e na análise de gestos humanos reais, de forma a garantir que a posição dos dedos e a orientação da mão transmitam a sensação de realismo e coerência com o objeto interativo. Quando um jogador pode interagir com um objeto longo, onde as posições das mãos podem variar, foram implementadas poses dinâmicas que se ajustam à posição do momento de interação.

Para estas poses, foram introduzidas cópias dos modelos das mãos (esquerda e direita) com a pose desejada, que foram posicionadas no espaço de forma a coincidir com a posição do objeto interativo. Ao interagir com o objeto, o modelo da mão do jogador é substituído pelo modelo da pose correspondente, criando a ilusão de que a mão do jogador se molda ao objeto. Um exemplo desta abordagem é a pose para rodar o botão rotativo do gerador, ilustrada na Figura 5.7.



Figura 5.7: Pose para rodar o botão rotativo do gerador.

Botões

Cada botão interativo contém um *script* dedicado (`ButtonPress.cs`) que define o seu comportamento. O objetivo principal deste *script* é despoletar eventos e fornecer *feedback* visual e sonoro ao utilizador quando o botão é pressionado.

A deteção da interação é feita através dos eventos `hoverEntered` e `hoverExited`, disponibilizados pelo `XRBaseInteractable` do **XR Interaction Toolkit**. Estes eventos são despoletados sempre que um interator entra ou sai da zona de Estes eventos são despoletados sempre que um interator entra ou sai da zona de deteção do botão, respetivamente. O *script* verifica ainda se o interator responsável pela interação é do tipo `XR Poke Interactor`, garantindo que apenas interações por toque (*poke*) acionam o comportamento do botão, e não outros tipos de interação, como o *grab* ou o *ray interaction*.

O *feedback* visual é dado pelo próprio movimento físico do botão. Quando o `XR Poke Interactor` entra em contacto com o botão, a sua posição é recalculada a cada frame com base na projeção do deslocamento do interator sobre um eixo local definido (`localAxis`), de forma a simular o percurso real de um botão físico a ser premido. Este deslocamento é limitado por um valor máximo (`maxTravel`), que corresponde à distância que o botão pode

percorrer antes de ser considerado totalmente pressionado. Quando o interator deixa de estar em contacto com o botão, este regressa suavemente à sua posição inicial através de uma interpolação linear (**Lerp**), controlada pela variável **releaseSpeed**.

O *feedback* sonoro é acionado no instante em que o botão atinge o seu deslocamento máximo, através da reprodução de um **AudioClip** associado, utilizando um **AudioSource** configurado no Inspetor do Unity.

Cada botão possui ainda um **UnityEvent** público (**buttonAction**), que pode ser configurado no inspetor para acionar diferentes ações consoante o contexto de utilização do botão, como, por exemplo, controlar o elevador. Este evento é invocado apenas uma vez por cada pressão completa, evitando disparos repetidos enquanto o botão permanece pressionado.

Botões Rotativos (*Knob/Dial*)

Para além dos botões tradicionais, o laboratório inclui também botões rotativos (*knobs*), utilizados para interações que requerem um ajuste contínuo ou gradual de um valor, como no gerador. Este comportamento é implementado através do script **KnobRotator.cs**.

A interação é iniciada quando o jogador agarra o objeto através de um **XRGrabInteractable**, recorrendo aos eventos **selectEntered** e **selectExited** do **XR Interaction Toolkit**. Ao ser agarrado, o script regista o interator responsável e passa a monitorizar, em cada frame, a rotação do controlador do jogador em torno do eixo Z, comparando-a com um ângulo de referência inicial.

Sempre que a diferença entre o ângulo atual e o ângulo de referência ultrapassa uma margem de tolerância (**angleTolerance**), o *dial* associado (**linkedDial**) é rodado num incremento fixo (**snapRotationAmount**), simulando um movimento de “*encaixe*” (*snapping*) típico de botões rotativos físicos, em vez de uma rotação contínua e livre. É também tratado o caso particular em que a rotação do controlador ultrapassa a fronteira entre 360° e 0°, de forma a evitar que essa transição seja interpretada como uma rotação inversa indevida.

Cada incremento de rotação é acompanhado de *feedback* sonoro, de forma a reforçar a perceção do utilizador de que a interação teve efeito. Adicionalmente, o script expõe um evento (**OnValueChanged**), invocado sempre que o valor do botão rotativo é alterado, permitindo que outros sistemas do laboratório reajam a essas mudanças, como a atualização de um valor apresentado no painel do gerador.

Alavancas

As alavancas são elementos interativos definidos apenas por duas posições discretas: ativada e desativada. A sua implementação baseia-se na utilização de dois **XRSocketInteractor**, que representam as posições extremas da alavanca. Para detetar a interação, basta ler os eventos **selectEntered** e **selectExited** de cada *socket*, de forma a determinar quando a alavanca é movida para uma das posições.

Elevador

O elevador é um elemento interativo que permite ao jogador deslocar-se verticalmente entre diferentes pisos do laboratório. A sua implementação baseia-se em estados de animação, acionados pelos botões, como referido anteriormente (5.4.1 - Botões).

O elevador possui um *script* dedicado (**ElevatorController.cs**), que expõe os métodos públicos **GoUp** e **GoDown**, invocados pelos eventos dos botões correspondentes. O ciclo da máquina de estados resume-se a três estados principais: parado, a subir e a descer. A transição entre estes estados é controlada pelos eventos dos botões, que verificam se o elevador se encontra parado (**isMoving**) antes de iniciar qualquer movimento.

O funcionamento do elevador está ainda condicionado ao estado do gerador elétrico do laboratório: o *script* subscrive os eventos **OnEnergyEstablished** e **OnEnergyLost**, disponibilizados pelo **GeneratorScript**, de forma a impedir a ativação do elevador sempre que não exista energia estabelecida no sistema. Este mecanismo reforça a coerência do ambiente, uma vez que o elevador só pode ser utilizado depois de o gerador ser ligado pelo jogador.

A animação do elevador é gerida por um **Animator**, que reproduz a sequência de subida ou descida através dos *triggers* **ElevatorUp** e **ElevatorDown**, sincronizando o movimento físico do elevador com a animação visual. O regresso ao estado parado é assinalado através do método **OnArrived**, invocado por um evento de animação no momento em que o elevador chega ao piso de destino.

Quanto aos efeitos visuais e sonoros, são utilizados eventos de animação para disparar sons de motor (**PlayMoveSound**) e de paragem (**PlayStopSound**), bem como um sistema de partículas de fumo (**PlaySmoke/StopSmoke**), de forma a reforçar o realismo e a imersão da experiência.

Para evitar problemas de movimentação brusca ou colisões com o jogador, o elevador possui um **Collider** definido como *Trigger* (**ElevatorTrigger.cs**), que deteta a entrada e saída do jogador através da respetiva *tag*. Ao entrar na zona do elevador, o jogador é tornado filho do **transform** do elevador, garantindo que se desloca solidariamente com a plataforma durante o movimento; adicionalmente, o seu **Rigidbody** é definido como *kinematic*, impedindo que o sistema de física interfira com a translação do elevador. Ao sair da zona do elevador, o processo é revertido: o jogador deixa de ser filho do elevador e o seu **Rigidbody** volta a ser controlado normalmente pela física do motor de jogo.

Portas Automáticas

Ao longo do laboratório existem várias portas automáticas que se abrem e fecham em resposta à presença do jogador. A implementação destas portas baseia-se na utilização de **BoxColliders** definidos como *Trigger*, que detetam a entrada e saída do jogador através da respetiva *tag*. Quando o jogador entra na zona de deteção, a porta inicia uma animação de abertura, reproduzindo também um efeito sonoro correspondente. Ao sair da zona, a porta fecha-se automaticamente, garantindo que o jogador não consegue atravessar a porta sem que esta esteja aberta.

Estas estão bloqueadas no início do jogo, sendo desbloqueadas a partir do momento em que o jogador conclui o Puzzle 1, restabelecendo a energia do laboratório. O comportamento destas portas é gerido por um *script* dedicado, o **DoorController**, responsável por controlar a animação, o som e o estado de bloqueio de cada porta. Este componente subscrive os eventos estáticos **OnEnergyEstablished** e **OnEnergyLost**, expostos pelo **GeneratorScript**, permitindo que todas as portas do laboratório reajam de forma sincronizada ao restabelecimento ou perda de energia, sem necessidade de referências diretas entre objetos.

Enquanto a energia não estiver estabelecida, as chamadas a **OpenDoor** são ignoradas, mantendo a porta bloqueada independentemente da deteção do jogador. Após a conclusão

do Puzzle 1 e o consequente restabelecimento da energia, a variável `_energyEstablished` é atualizada para `true`, permitindo que a porta responda normalmente aos eventos de entrada e saída do jogador na zona de detecção, através dos métodos `OpenDoor` e `CloseDoor`.

Escadotes e Escalada

A escalada é um dos sistemas interativos que já vêm implementados no `XR Interaction Toolkit`, através de dois componentes que trabalham em conjunto: o `Climb Interactable` e o `Climb Provider`. Estes componentes permitem que o jogador interaja com objetos escaláveis, como escadas ou escadotes, permitindo-lhe subir e descer.

O `Climb Interactable` é colocado no objeto que se pretende tornar escalável (por exemplo, os degraus de uma escada) e estende a classe base `XRBaseInteractable`, utilizando a interação de seleção para conduzir o movimento do jogador: enquanto o jogador mantiver o objeto selecionado (agarrado) através de um `Interactor`, o seu movimento é traduzido em deslocação do jogador no mundo virtual. Por definição, este componente exige um `Rigidbody` no objeto e tem o modo de seleção (*Select Mode*) configurado para `Multiple`, o que permite, por defeito, a escalada com as duas mãos em simultâneo. É ainda possível associar a cada `Climb Interactable` um conjunto de definições próprio (*Climb Settings Override*), que substitui, apenas para esse objeto, as restrições de movimento definidas globalmente no `Climb Provider`, uma prática comum, por exemplo, nos travessões superiores de uma escada, para permitir movimento livre nos eixos horizontais e possibilitar que o jogador saia da escada para uma plataforma no topo.

O `Climb Provider`, por sua vez, é colocado na `XR Origin` e é responsável por efetivamente mover o jogador. Este desloca a `XR Origin` de forma inversa ao movimento do `Interactor` que está a selecionar o `Climb Interactable`, de forma semelhante ao efeito visual do "puxar uma corda" para se deslocar. Se existir mais do que um `Interactor` a selecionar o mesmo objeto, apenas o mais recente influencia o movimento. Este tipo de locomoção é conceptualmente semelhante ao movimento por agarrar (*grab movement*), mas está restrito aos objetos que possuam explicitamente um `Climb Interactable`, ao contrário do *grab movement*, que pode ser aplicado a qualquer superfície. As restrições de movimento por eixo (local ao objeto) podem ser configuradas ao nível do `Climb Provider` ou sobrepostas individualmente por cada `Interactable`, permitindo um melhor controlo sobre o comportamento da escalada em diferentes contextos do jogo.

5.4.2 Implementação da Lógica dos Puzzles

Nesta subsecção, é detalhada a implementação da lógica dos puzzles, incluindo a gestão de estados, eventos e interações com os elementos do jogo, descritos anteriormente.

Puzzle 1: Restabelecimento de Energia

A implementação do Puzzle 1 baseia-se na validação sequencial de estados e no despoleamento de eventos associados ao gerador elétrico. O comportamento deste mecanismo é gerido por um *script* dedicado que expõe dois métodos públicos principais:

- `AddFuse()`: Ativa o gerador e dispara o evento `onGeneratorOn`;
- `RemoveFuse()`: Desativa o gerador e dispara o evento `onGeneratorOff`, repondo o estado de energia não estabelecida.

Para iniciar o gerador, é necessário inserir um fusível no respetivo recetáculo. Para facilitar o alinhamento do objeto, foi utilizado o componente `XRSocketInteractor` do `XR Interaction Toolkit`, que posiciona automaticamente o fusível no local correto através da funcionalidade de *snap*. A ligação à lógica do puzzle é feita através dos eventos nativos `selectEntered` e `selectExited`, que chamam os métodos `AddFuse()` e `RemoveFuse()` quando o fusível é inserido ou removido.

Depois de validada a presença do fusível, é ativado um ecrã digital que apresenta a tensão do gerador, inicialmente igual a 0 V. Para alterar este valor, o jogador interage com um botão rotativo, que aumenta a tensão incrementalmente, de acordo com a mecânica descrita na sub-secção anterior. Cada alteração de tensão é propagada através de um evento, ao qual o *script* do gerador se encontra subscrito para evitar qualquer dependência direta entre os componentes.

Quando a tensão atinge os 600 V, o sistema reproduz um som para confirmar que o valor está correto. O último passo consiste em baixar uma alavanca, cujo funcionamento foi também implementado com um `XRSocketInteractor`. Quando a alavanca alcança a posição inferior, o puzzle fica concluído, a energia é restabelecida no laboratório e o evento `onEnergyEstablished` é disparado.

Para indicar o estado do puzzle sem recorrer a elementos de interface adicionais, foi implementado um sistema de *feedback* integrado no ecrã do gerador. A cor da interface e da iluminação altera-se de forma gradual através da função `Color.Lerp`, em função da diferença entre a tensão atual e o valor pretendido de 600 V. Quando a tensão entra numa zona crítica, a iluminação e a opacidade do texto passam a oscilar com base numa função sinusoidal dependente do tempo.

O sistema inclui também *feedback* sonoro. Cada alteração da tensão reproduz um som, que é substituído por um sinal diferente quando o valor de 600 V é alcançado. Desta forma, o jogador recebe confirmação do progresso sem necessidade de manter atenção constante ao ecrã, o que favorece a interação com os elementos físicos do painel.

Depois de concluir esta secção introdutória, as luzes do laboratório são ligadas, as portas automáticas são ativadas (cf. 5.4.1) e o elevador fica disponível para o jogador.

Puzzle 2: Formas e Números

O segundo puzzle do jogo desafia o jogador a relacionar objetos físicos com informações apresentadas num terminal digital. Cada objeto deve ser comparado com um peso com um número associado, para descobrir o código de saída. A implementação deste puzzle baseia-se na interação física entre objetos `Rigidbody` e na validação de estados através de eventos. O objeto central, uma balança, é implementada via `Joints` para obter o comportamento físico desejado.

Para o teclado, foi implementado o componente `Keypad`, responsável por gerir a introdução do código numérico por parte do jogador. Cada tecla pressionada invoca o método `OnKeyPressed`, que concatena o carácter correspondente a uma *string* interna (`_inputKeyCode`) e atualiza o ecrã digital através de um componente `TMP_Text`. O jogador pode ainda remover o último caractere introduzido (`OnClearPressed`) ou submeter o código para validação (`OnEnterPressed`). A validação é realizada em `VerifyKeyCode`, que compara a *string* introduzida com o valor de `keyCode`, definido previamente no Inspector do Unity. Caso a comparação seja bem-sucedida, o puzzle é assinalado como concluído através da chamada a `PuzzleManager.Instance.CompletePuzzle(puzzleKey)`, notificando o gestor central de puzzles do estado atualizado. Em caso de falha, a *string*

introduzida é reiniciada, obrigando o jogador a repetir a introdução do código.

Cada botão do teclado possui o componente **ButtonPress**, descrito anteriormente, que deteta a interação do jogador e invoca o método correspondente no **Keypad**, seja este um valor numérico ou um *clear*.

Puzzle 3: Labirinto Digital

O terceiro puzzle inicia-se quando o jogador aponta para um feixe de transferência, identificado no código através da *tag* "Transfer" e detetado por um *raycast* contínuo lançado a partir do olhar do jogador (**TransferBeam**). Ao detetar a colisão, é disparado um efeito de transição visual e, decorrido o tempo desse efeito, o jogador é reposicionado de acordo com valores predefinidos num *ScriptableObject* (**PlayerTransferData**). É esta mudança de escala que sustenta, a nível narrativo e mecânico, a passagem do jogador para uma versão miniaturizada e “digital” de si próprio dentro do labirinto.

A entrada no labirinto despoleta também o inimigo que o jogador terá de evitar. O **PuzzleMazeManager** funciona como ponto de entrada desta fase: ao ser notificado do início do puzzle, ativa a máquina de estados do inimigo (**EnemyStateMachine**), que alterna entre três estados: *Idle*, *Wander* e *Run*. O inimigo permanece parado durante um intervalo inicial, após o qual passa a vaguear pelo labirinto nas imediações do jogador, recorrendo à *NavMesh* do Unity para se deslocar. Enquanto vagueia, um segundo *raycast*, lançado a partir do próprio inimigo, verifica se este tem o jogador diretamente à sua frente; caso o detete, transita para o estado de perseguição, aumentando a sua velocidade de deslocação.

O comportamento e os parâmetros de cada inimigo, como a velocidade, dano de ataque, alcance de ataque e mordida, bem como os conjuntos de sons associados a cada estado, estão isolados num *ScriptableObject* próprio (**ZombieData**), separando a configuração da lógica de comportamento e permitindo reutilizar a mesma máquina de estados para diferentes variantes de inimigo. A resposta sonora é gerida de forma independente por inimigo através do **ZombieSoundManager**, que lê o estado atual diretamente da máquina de estados associada a essa instância específica, evitando que o som de um inimigo seja influenciado pelo estado de outro caso existam vários em simultâneo no labirinto.

5.4.3 Gestão da Narrativa

A narrativa do jogo funciona como uma série de diálogos e mensagens apresentadas ao jogador em momentos especiais, como ao iniciar ou concluir um puzzle. O sistema funciona à base de *triggers* colocados em pontos estratégicos do mapa. Estes *triggers* narrativos são compostos por um ficheiro de áudio e um componente **AudioSource**, que reproduz o áudio quando o jogador entra na zona de deteção. Estes *triggers* só serão ativados consoante a sua progressão no jogo, de forma a evitar que o jogador ouça mensagens fora de contexto.

5.4.4 Gestão do Progresso do Jogo

Para permitir uma melhor experiência de utilizador, foi implementado um sistema para guardar e carregar o progresso do jogo. Este sistema divide-se em três partes: a posição do jogador, o estado dos puzzles e a posição de todos os objetos interativos.

A classe central deste sistema é o **GameDataManager**, que segue o padrão **Singleton**, demonstrado no excerto de código 5.1, garantindo que existe apenas uma instância ativa em toda a aplicação e que esta persiste entre cenas.

Código 5.1: `GameManager` - Singleton implementado em C#.

```

1  public static GameManager Instance { get; private set; }
2  ...
3  private void Awake()
4  {
5      if (Instance != null && Instance != this)
6      {
7          Destroy(gameObject);
8      }
9      else
10     {
11         Instance = this;
12         DontDestroyOnLoad(gameObject);
13     }
14 }

```

Quando é desencadeada uma gravação, esta classe recolhe o estado atual do jogo, que inclui a posição do jogador, puzzles concluídos e transformações dos objetos móveis, e serializa-o em formato JSON através do `JsonUtility` do Unity, e escreve o resultado num ficheiro no caminho dado por `Application.persistentDataPath`, uma localização gerida pelo sistema operativo que persiste entre sessões de jogo. O carregamento segue o processo inverso: o ficheiro é lido e os valores são restaurados diretamente nos respetivos componentes.

De notar que o estado dos puzzles é internamente representado como um `Dictionary<string, bool>`, mas é guardado em dois `List` paralelos, um para as chaves e outro para os valores. Esta conversão deve-se ao facto de o `JsonUtility` não suportar a serialização direta de dicionários, sendo a estrutura reconstruída no momento do carregamento.

Para identificar quais os objetos cujo estado deve ser guardado, foi criado o componente `SaveableObject`. Este componente é adicionado a qualquer objeto interativo que necessite de persistência e atribui-lhe um identificador único (GUID). Os objetos registam-se automaticamente numa coleção estática global no momento em que são criados, e removem-se quando são destruídos. Durante o carregamento, o sistema percorre esta coleção e restaura a posição e rotação de cada objeto com base no seu identificador, de forma independente da ordem ou hierarquia da cena.

De modo a automatizar a configuração inicial, foi desenvolvido uma ferramenta de editor acessível através do menu `Tools > Setup Saveable Objects`, exemplificado na Figura 5.8. Esta ferramenta percorre toda a cena em busca de objetos com comportamentos interativos, nomeadamente objetos com `XRGrabInteractable` ou com `Rigidbody` não cinemático, e adiciona-lhes automaticamente o componente `SaveableObject`. Este procedimento elimina a necessidade de configurar manualmente cada objeto da cena, que é mais trabalhoso e propício a erros de omissão.

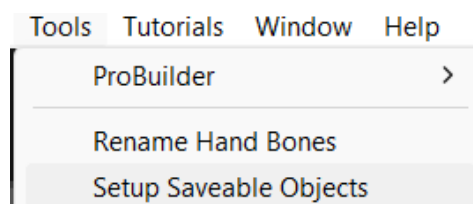


Figura 5.8: Janela de configuração inicial dos objetos interativos.

Por fim, para dar *feedback* ao utilizador após uma gravação, foi implementada uma animação de texto em HUD que surge, pisca três vezes com um *fade*, e desaparece automaticamente.

Capítulo 6

Validação e Testes

Este capítulo descreve o processo de validação do jogo junto de utilizadores, incluindo o instrumento de recolha de dados utilizado, o procedimento de teste e a análise estatística realizada aos resultados obtidos.

O capítulo está dividido em cinco secções. A secção 6.1 apresenta os objetivos dos testes de validação. A secção 6.2 descreve o questionário utilizado, baseado no GUESS-18, e as adaptações feitas ao mesmo. A secção 6.3 caracteriza a amostra e o procedimento seguido durante as sessões de teste. Na secção 6.4 é detalhada a abordagem de análise estatística adotada, e a secção 6.5 apresenta e discute os resultados obtidos. Por fim, a secção 6.6 identifica as principais limitações deste processo de validação.

6.1 Objetivos

Os testes de validação tiveram como objetivo avaliar a experiência do jogador em três dimensões principais: a compreensão das mecânicas e objetivos do jogo, a satisfação geral com a experiência (usabilidade, imersão, estética visual e sonora, entre outras) e a deteção de eventuais problemas de *design* que possam comprometer a jogabilidade, como pontos de confusão na navegação ou *puzzles* pouco claros.

6.2 Instrumento de Recolha de Dados

Para avaliar a satisfação do jogador, recorreu-se ao GUESS-18 [Keebler et al., GUESS-18, 2016], uma versão reduzida de 18 items da *Game User Experience Satisfaction Scale* (GUESS) [Phan, Keebler & Chaparro, 2016, Human Factors]. O GUESS-18 organiza-se em nove subescalas (duas perguntas cada), que cobrem aspetos como usabilidade, narrativa, imersão, prazer, liberdade criativa, estética sonora, gratificação pessoal, ligação social e estética visual. As respostas são recolhidas numa escala de Likert de 7 pontos, de “*Disagree Completely*” a “*Agree Completely*”, com um item de formulação negativa (“*I feel bored while playing the game*”) que é invertido antes da agregação por subescala.

Uma vez que o GUESS-18 não avalia diretamente a compreensão dos objetivos e mecânicas do jogo, o questionário foi complementado com um segundo bloco de perguntas, desenvolvido especificamente para este projeto. Este bloco foca-se em momentos-chave da experiência (por exemplo, perceber que era necessário abrir a porta para começar a andar, ou identificar as pistas necessárias para resolver o segundo *puzzle*) e utiliza uma escala de Likert de 5 pontos (*Strongly Disagree* a *Strongly Agree*), com um ponto central neutro

(*Undecided*). Ao contrário do GUESS-18, este bloco não foi psicometricamente validado, pelo que os seus resultados devem ser interpretados de forma mais qualitativa.

6.3 Amostra e Procedimento

Para este estudo foram feitos testes com um total de 8 participantes, com idades compreendidas entre os 20 e os 25 anos, com diferentes níveis de experiência com jogos e realidade virtual. Os participantes foram selecionados com alguns alunos do ISEL e amigos pessoais, que não tinham participado no desenvolvimento do projeto, e foram convidados a experimentar o jogo e a preencher o questionário de avaliação.

6.4 Análise Estatística

As respostas ao questionário foram codificadas numericamente e analisadas separadamente por bloco, dado utilizarem escalas distintas. O bloco do GUESS-18 (7 pontos) e o bloco de compreensão desenvolvido para este projeto (5 pontos). Em ambos os casos, a opção central, “*No Opinion*” no GUESS-18 e “*Undecided*” no bloco de compreensão, foi tratada de forma distinta, apenas a segunda corresponde a um verdadeiro ponto neutro da escala, pelo que a primeira foi tratada como dado em falta em vez de valor numérico central.

Dada a natureza ordinal dos dados, optou-se por reportar medianas e modas por item, em vez de médias, complementadas com gráficos de barras divergentes que mostram a distribuição completa das respostas por categoria. Este tipo de gráfico permite visualizar simultaneamente a concentração de respostas positivas e negativas, bem como a proporção de respostas em falta.

Devido à dimensão reduzida da amostra recolhida nesta fase do projeto, optou-se por não realizar testes de inferência estatística (por exemplo, testes de Mann-Whitney), uma vez que o poder estatístico destes testes seria insuficiente para produzir resultados fiáveis. A análise foi por isso conduzida como um estudo exploratório/piloto, com o objetivo de identificar tendências e possíveis pontos de melhoria a validar em futuras iterações com uma amostra mais alargada.

O processamento e visualização dos dados foi feito através de um *script* em Python, utilizando as bibliotecas `openpyxl` para leitura das respostas, `NumPy` para o cálculo das estatísticas descritivas e `Matplotlib` para a geração dos gráficos.

6.5 Resultados

A trabalhar nisso ...

6.6 Limitações

A principal limitação deste processo de validação prende-se com a dimensão reduzida da amostra, o que impede a generalização estatística dos resultados e limita a análise a um nível descritivo/exploratório. Adicionalmente, o bloco de perguntas sobre compreensão do jogo, por não ser um instrumento validado, deve ser interpretado com maior cautela do que os resultados do GUESS-18.

Capítulo 7

Utilização de Inteligência Artificial

Este capítulo detalha o uso de Inteligência Artificial (IA) utilizada ao longo do desenvolvimento do projeto. A IA foi utilizada majoritariamente para correção de escrita, e esclarecimento de dúvidas sobre a organização do relatório. Quanto ao jogo, a IA foi apenas utilizada para esclarecimento de dúvidas sobre a implementação, não tendo sido utilizada para gerar código ou *assets*.

Capítulo 8

Conclusões e Trabalho Futuro

Durante o desenvolvimento do projeto, foi possível explorar e aplicar conceitos de realidade virtual, *design* de jogos e interação pessoa-máquina e integrá-los num ambiente virtual interativo. Ao criar todos os objetos e texturas do zero, foi possível desenvolver uma identidade visual consistente e coerente com a narrativa do jogo e possibilitou mais controlo sobre o desenvolvimento. No fim, foi possível implementar várias soluções de interação, como o sistema de poses dinâmicas, botões, botões rotativos, alavancas, elevador e portas automáticas, todas interativas a partir de movimentos reais.

A escolha do `OpenXR` como base de desenvolvimento, através do `XR Interaction Toolkit`, revelou-se acertada, visto que permitiu testar o jogo tanto no `Meta Quest` como no `HTC Vive Pro` sem necessidade de manter duas versões separadas do projeto, e disponibilizou já resolvidos sistemas de interação de complexidade considerável, como a escalada, que de outra forma teriam exigido um esforço de implementação maior. A abordagem iterativa seguida ao longo do projeto, com recurso a *greyboxing* nas fases iniciais antes de avançar para os elementos gráficos finais, permitiu corrigir vários problemas de disposição espacial e navegação que só se tornam evidentes quando experienciados em primeira pessoa, dentro dos próprios óculos.

Os testes de validação realizados junto de utilizadores, com base numa adaptação do `GUESS-18`, confirmaram que aspetos como a atmosfera, os gráficos e o áudio do jogo foram bem recebidos, e não identificaram problemas graves de *motion sickness*. Por outro lado, também revelaram alguns pontos de atrito, nomeadamente na compreensão de certos controlos e na dificuldade de alguns participantes em interpretar as pistas do segundo puzzle, o que sugere que ainda há espaço para melhorar a comunicação visual de determinados mecanismos do jogo.

8.1 Trabalho Futuro

Uma das limitações mais evidentes deste trabalho foi a reduzida dimensão da amostra utilizada nos testes de validação. Um passo natural para dar continuidade ao projeto seria repetir estes testes com um número mais alargado de participantes, o que permitiria não só obter resultados estatisticamente mais robustos, como também recorrer a testes de inferência que não foram possíveis nesta fase, como comparações entre diferentes perfis de jogadores (por exemplo, com e sem experiência prévia em VR).

Ao nível do conteúdo do jogo, seria interessante expandir o número de puzzles e áreas exploráveis do laboratório, mantendo a filosofia de integração diegética que serviu de base

a este projeto, inspirada em títulos como “Half-Life: Alyx” e “Ghost Town”. O sistema modular de peças de ambiente desenvolvido facilita esta expansão, já que grande parte da infraestrutura necessária para construir novos espaços já está pronta a ser reutilizada.

Por fim, valeria a pena rever alguns dos mecanismos de interação apontados como menos intuitivos nos testes, como os controlos gerais do jogo e o sistema de pesagem do segundo puzzle, e considerar a inclusão de um breve tutorial inicial, sugestão que aliás foi deixada por um dos participantes nos testes de validação.

Bibliografia

[Unity, site] Unity Technologies. *Unity Official Website*. <https://unity.com/pt>

[XR Interaction Toolkit, Unity, site] Unity Technologies. *XR Interaction Toolkit Documentation*. <https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@3.0/manual/index.html>

[ProBuilder, Unity, site] Unity Technologies. *ProBuilder Documentation*. <https://docs.unity3d.com/Packages/com.unity.probuilder@4.0/manual/index.html>

[Neil Blevins, Greyboxing, site] Neil Blevins. *Greyboxing in Game Development*. http://www.neilblevins.com/art_lessons/greybox/greybox.htm

[Keebler et al., GUESS-18, 2016] The Journal of Usability Studies. *GUESS-18 Scoring Guidelines*. https://uxpajournal.org/wp-content/uploads/sites/7/pdf/JUS_Keebler_GUESS-18%20Scoring%20Guidelines.pdf

[Phan, Keebler & Chaparro, 2016, Human Factors] ResearchGate. *The Development and Validation of the Game User Experience Satisfaction Scale (GUESS)* https://www.researchgate.net/publication/308343588_The_Development_and_Validation_of_the_Game_User_Experience_Satisfaction_Scale_GUESS

[Six Degrees of Freedom, Wikipedia, site] Wikipedia contributors. *Six Degrees of Freedom*. https://en.wikipedia.org/wiki/Six_degrees_of_freedom

[Norman, 2013] Donald A. Norman. *The Design of Everyday Things*. Basic Books, Revised and Expanded Edition, 2013.

[Csikszentmihalyi, 1990] Mihaly Csikszentmihalyi. *Flow: The Psychology of Optimal Experience*. Harper & Row, 1990.

[Carson, Environmental Storytelling, site] Don Carson. *Environmental Storytelling: Creating Immersive 3D Worlds Using Lessons Learned from the Theme Park Industry*. Game Developer. <https://www.gamedeveloper.com/design/environmental-storytelling-creating-immersive-3d-worlds-using-lessons-learned-from>

[Jenkins, Spatial Stories] Henry Jenkins. *Game Design as Narrative Architecture*. 2004.